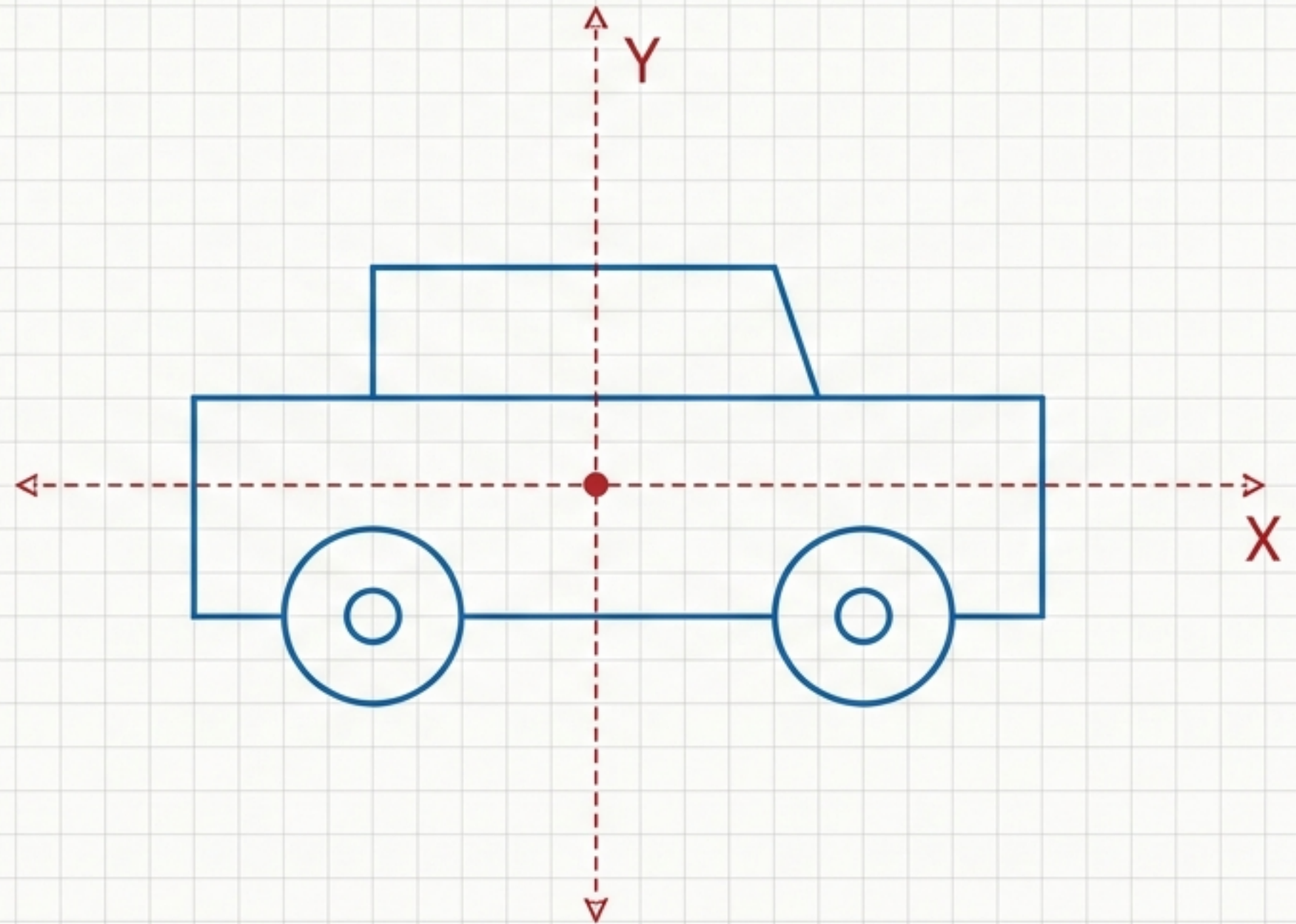
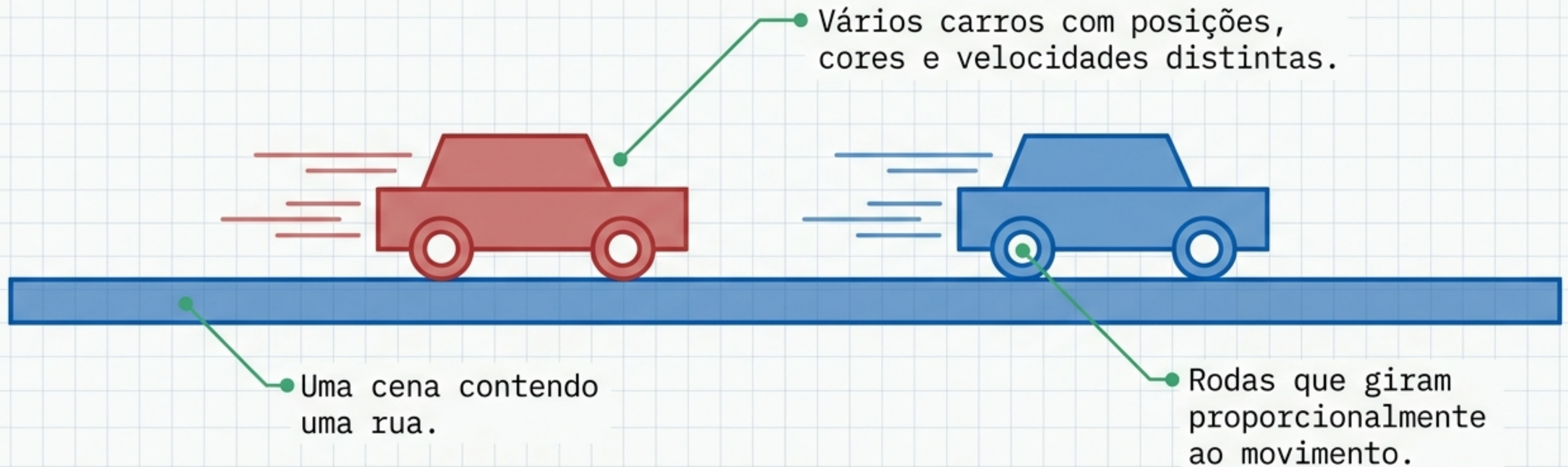


Modelagem e Animação Hierárquica

Integrando Programação Orientada a Objetos e Computação Gráfica

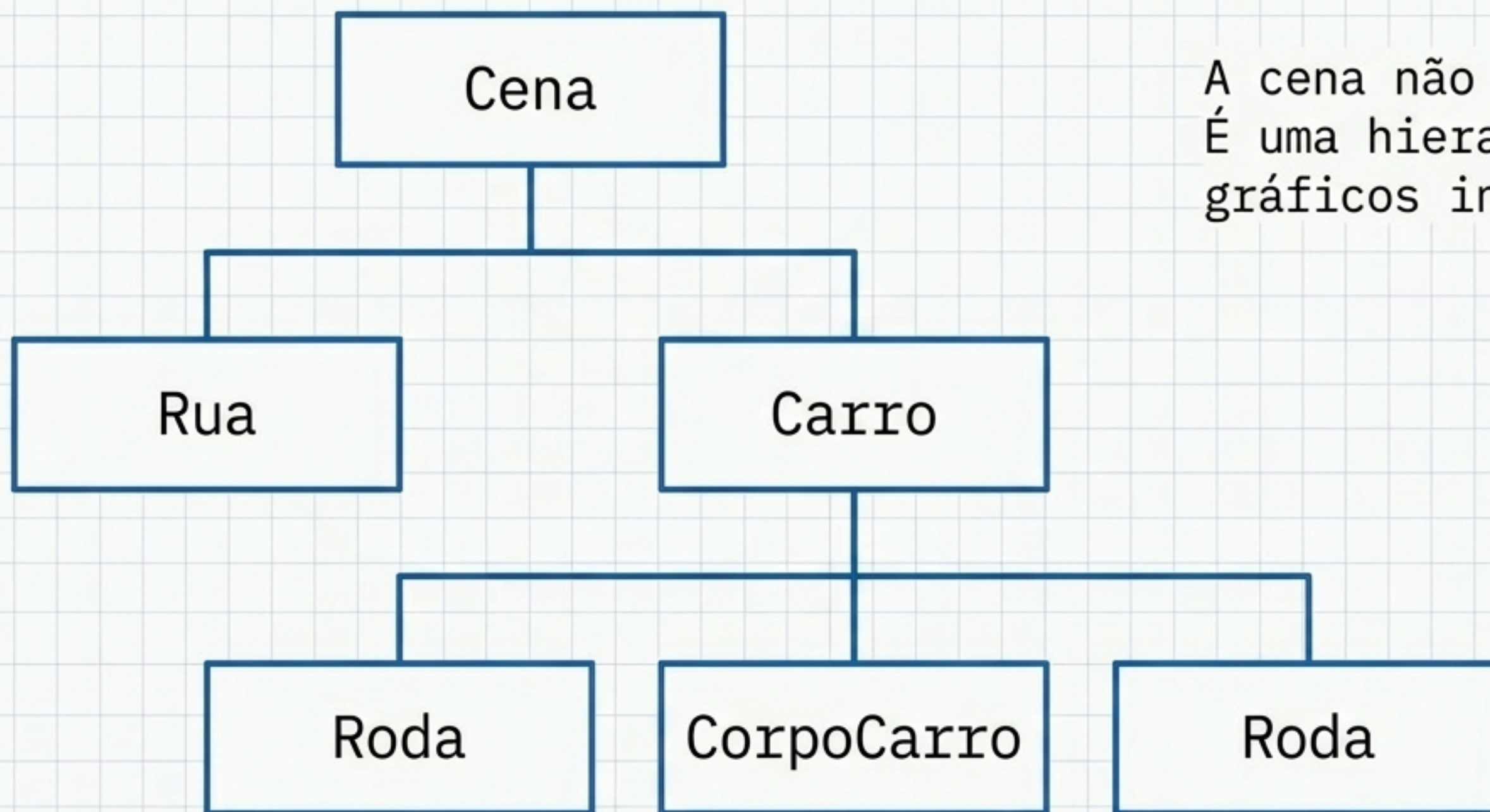


O objetivo: Simular um ambiente dinâmico com entidades independentes



O desafio central: Modelar os elementos da cena como objetos que possuem **estado**, **comportamento** e **representação gráfica**.

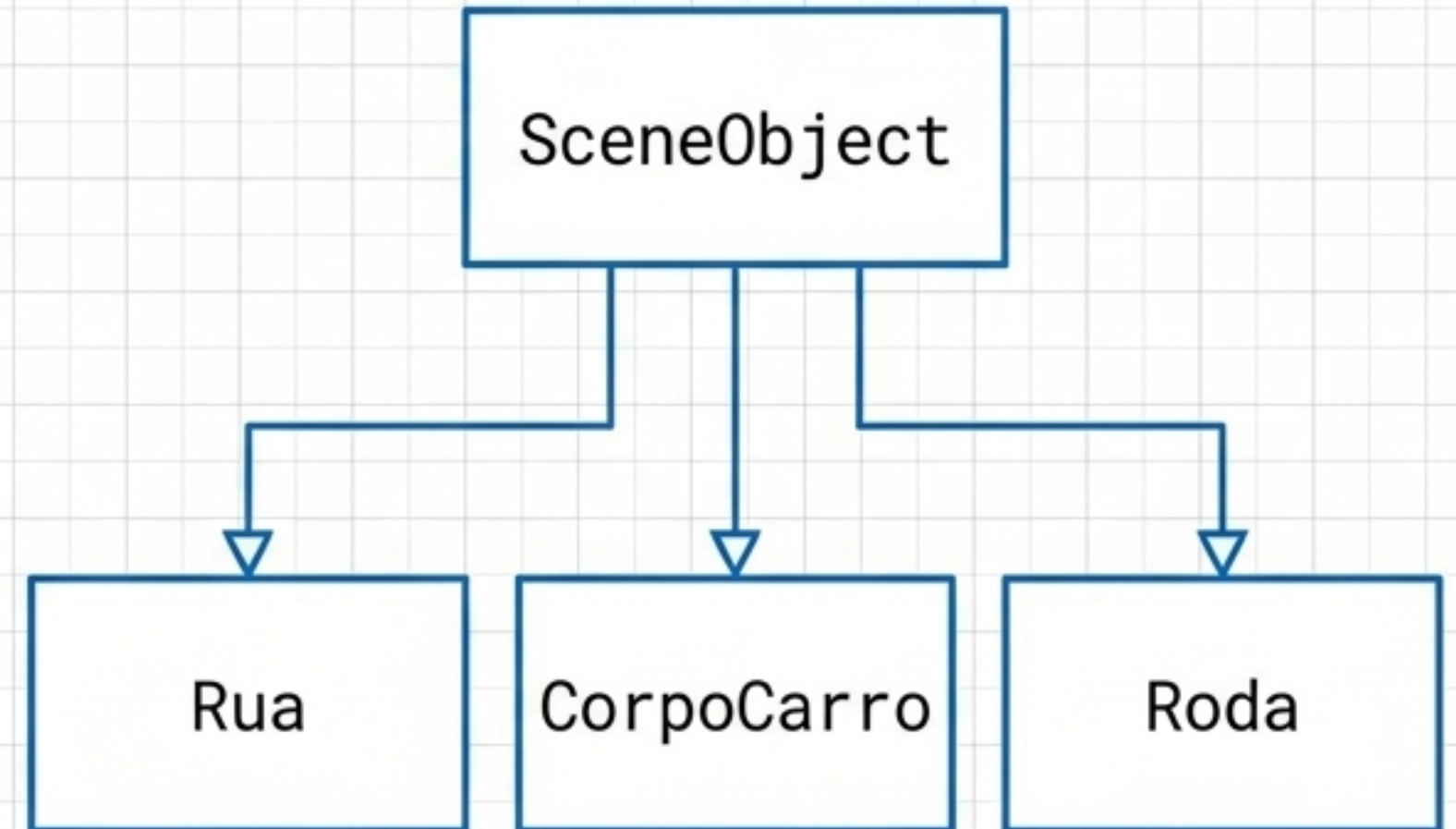
Da observação visual para a abstração estrutural



A cena não é uma imagem plana.
É uma hierarquia de objetos
gráficos interconectados.

A Fundação POO: Reutilização através da Herança

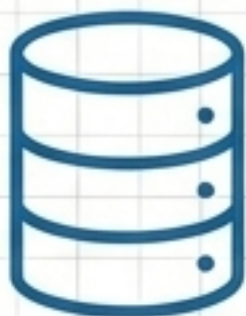
```
class SceneObject {  
    constructor(vertices, color) {  
        this.vertices = vertices;  
        this.color = color;  
        this.modelTransform = m3.identity();  
    }  
  
    atualizarTransformacao(modelTransform) { ... }  
}
```



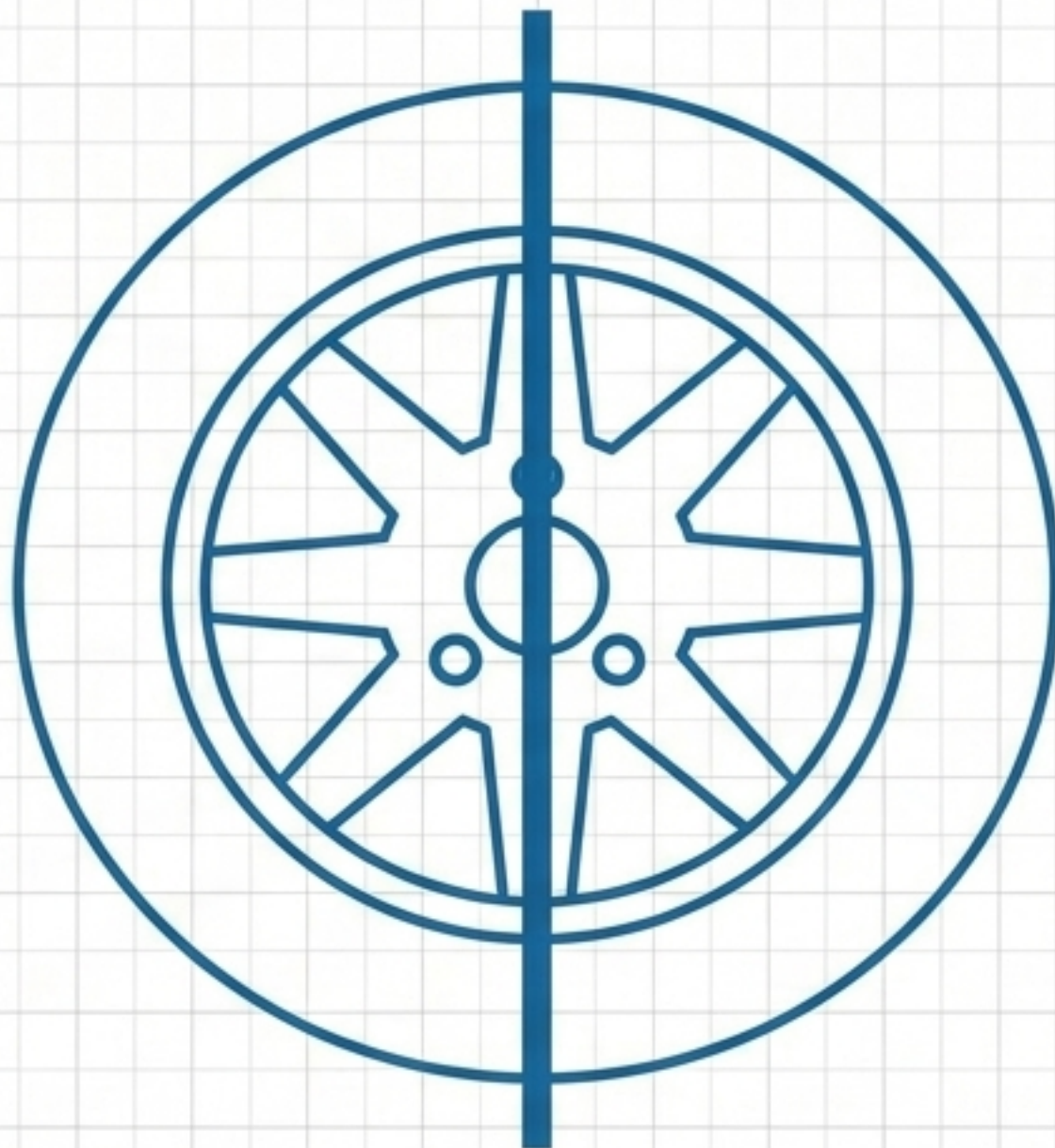
As subclasses reutilizam geometria (vertices), aparência (color) e posição no mundo (modelTransform) sem duplicar código.

O princípio do Encapsulamento: Estado e Comportamento

Estado
(O que o objeto tem)



posicaoX
scale
theta
angleSpeed



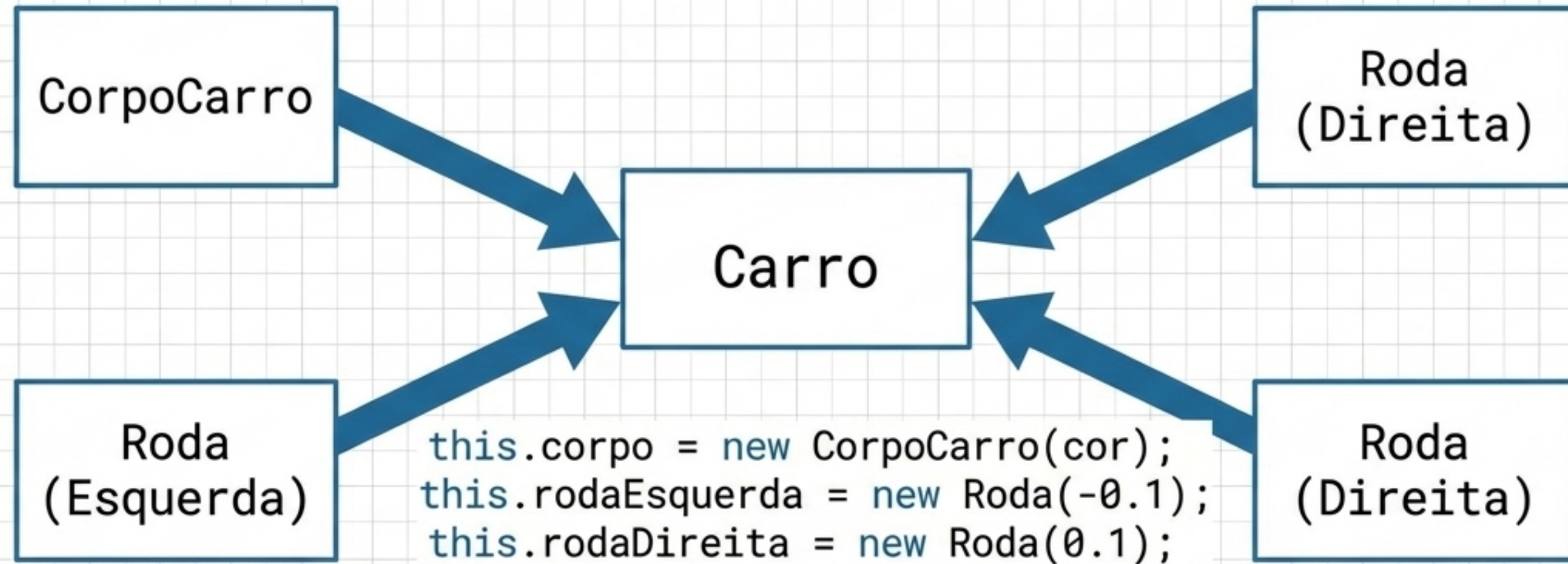
Comportamento
(O que o objeto faz)



girar(deltaTime)
atualizarTransformacao(..)

Um objeto é a união indivisível de dados e as operações que modificam esses dados.

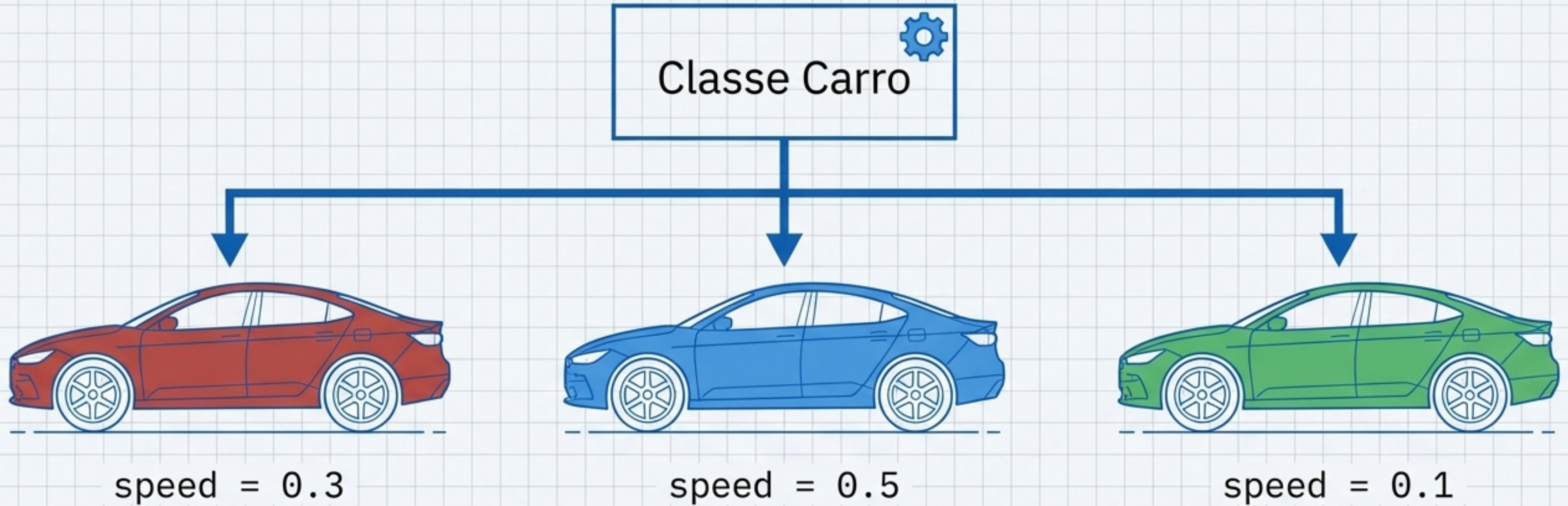
Construindo a hierarquia através da Composição



Herança (IS-A)
A Roda é um SceneObject. ↑

→ Composição (HAS-A)
O Carro tem uma Roda.

Instanciação: Um único molde, múltiplos estados independentes



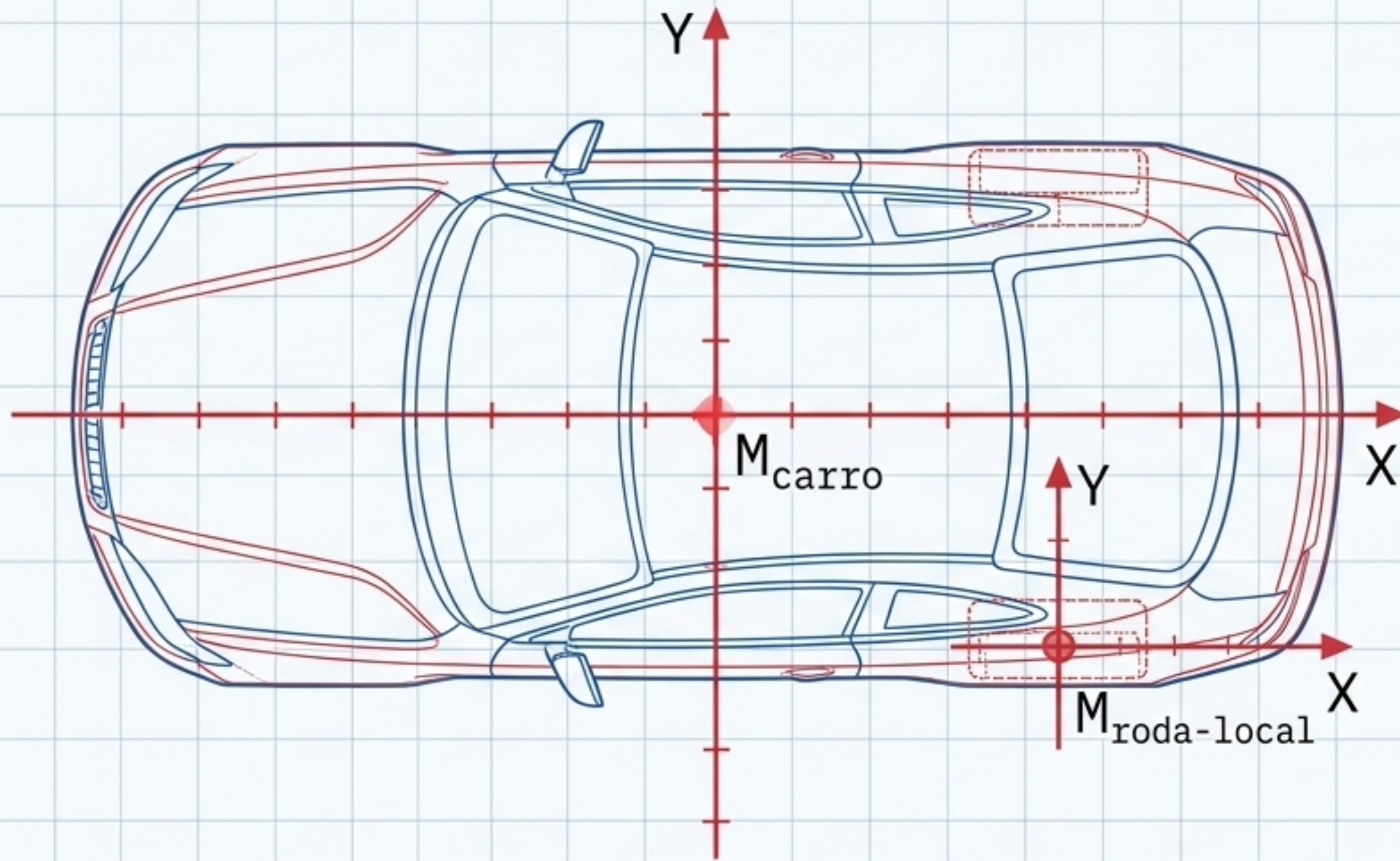
Isto evita criar CarroVermelho, CarroAzul.
A estrutura é universal; o estado é local.

O Abismo Gráfico: Separando o Objeto da Câmera

Model Transform		View Transform	
Pertence a:	Objeto (Carro, Rua, Roda)	Pertence a:	Cena / Câmera
A pergunta que responde:	"Onde estou no mundo?"	A pergunta que responde:	"Como o mundo é mapeado para a tela?"
Código:	<code>u_modelTransform</code>	Código:	<code>u_viewTransform</code>

$$P_{NDC} = M_{view} \times M_{model} \times P$$

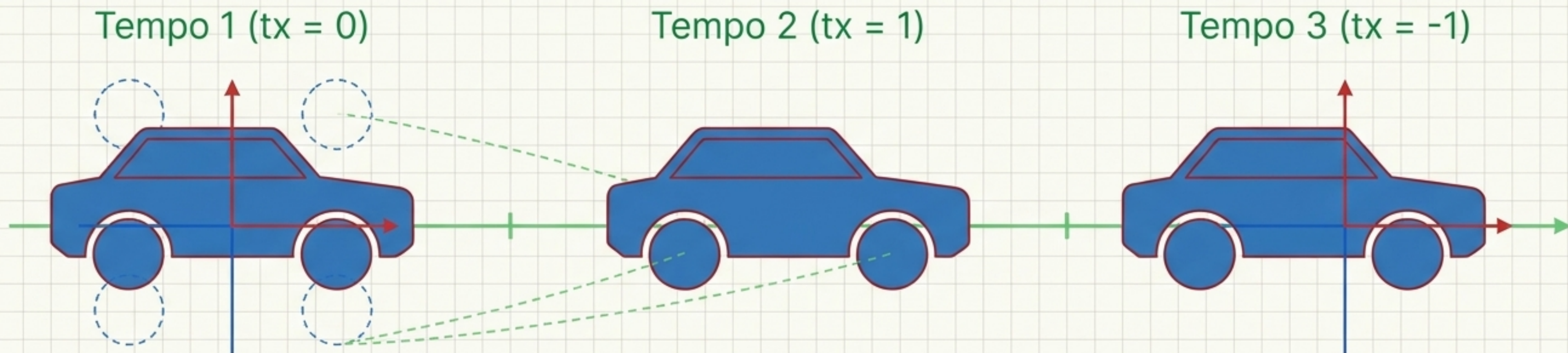
0 Foco: Matrizes em Transformações Hierárquicas



$$M_{\text{roda}} = M_{\text{carro}} \times M_{\text{roda-local}}$$

A roda não conhece sua posição absoluta no mundo. Ela conhece apenas sua posição relativa ao carro.

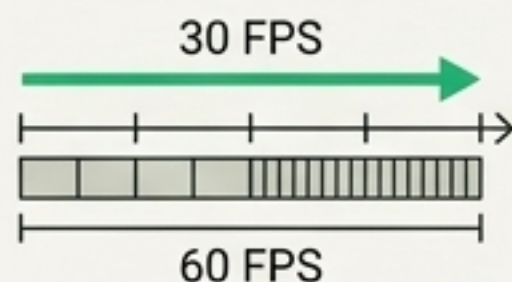
Por que a Hierarquia Importa? Propagação Automática



Não recalculamos manualmente a posição das rodas no mundo. Quando o carro se move, a matriz base é atualizada, e a transformação se propaga em cascata pela árvore de objetos.

A Física da Animação: Independência de Taxa de Quadros (Δt)

O Problema: 30 FPS vs 60 FPS não devem alterar a velocidade do jogo.

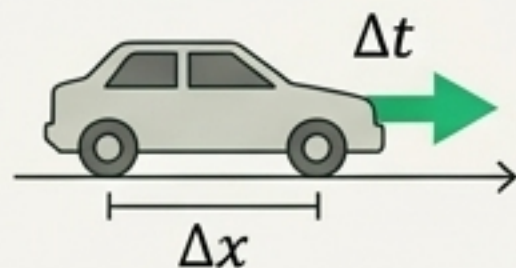


```
const dt = deltaTime / 1000; // Tempo em segundos
```

Movimento Translacional (Carro)

$$\Delta x = v \cdot \Delta t$$

(Posição = velocidade $\times$ tempo)



Movimento Rotacional (Roda)

$$\Delta \theta = \omega \cdot \Delta t$$

(Rotação = velocidade angular $\times$ tempo)



O movimento é ancorado no tempo real transcorrido, não na capacidade de renderização da máquina.

Separação de Responsabilidades: O Muro Arquitetural

Lógica da Aplicação (Scene / Carro)

- Calcular posições
- Atualizar estado
- Gerenciar hierarquia

Renderer, desenhe este objeto.

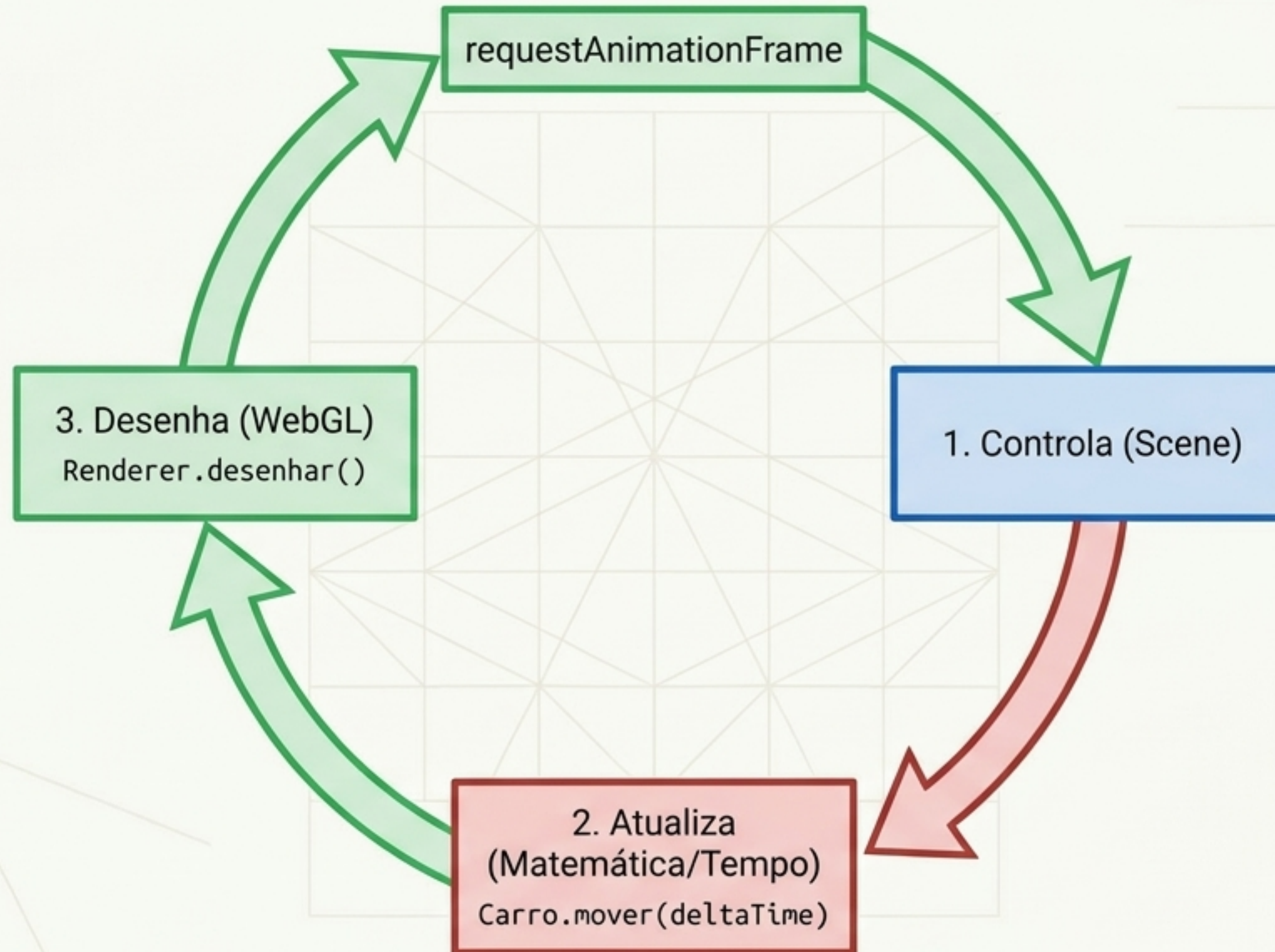
Lógica de Renderização (Renderer / WebGL)

- Comunicar com a GPU
- Vincular buffers
- Desenhar arrays

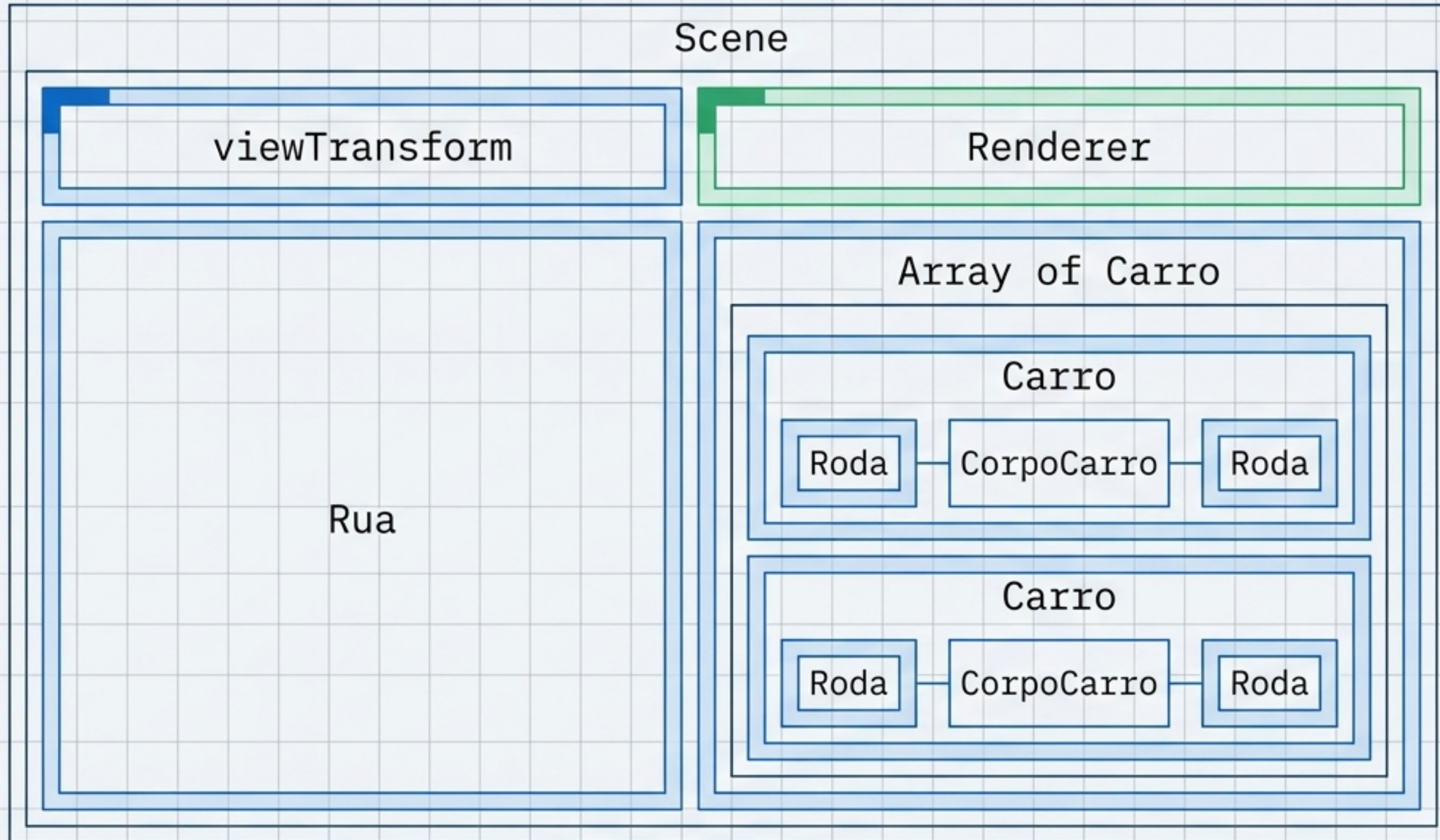
```
gl.bindBuffer(...)  
gl.drawArrays(...)
```

O Carro não precisa saber como desenhar pixels, apenas o que deve ser desenhado.

O Fluxo de Execução Contínua (Game Loop)



A Arquitetura Final do Sistema



A Grande Síntese: Onde as disciplinas se encontram

1. POO (Organiza)

Estrutura a aplicação através de hierarquias e encapsulamento.

2. Animação (Modifica)

Altera o estado matemático ao longo do tempo (Δt).

3. Comp. Gráfica (Materializa)

Transforma e desenha os objetos.

Scene → Carros → Corpo + Rodas

$$P_{NDC} = M_{view} \times M_{model} \times P$$

A integração perfeita entre arquitetura de código e resultado visual.